Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mcom functions

Synopsis

The functions prefixed with Mcom make up the Industrial Communication module. The Industrial Communication module allows you to communicate with a device or a robot controller using the specified industrial protocols.

Functions

McomAlloc
McomControl
McomFree
McomGetHookInfo
McomHookFunction
McomInquire
McomRead
McomSendPosition
McomWaitPositionRequest
McomWrite

McomAlloc

Synopsis

Allocate an Industrial Communication context.

Syntax

```
AIL_ID McomAlloc(
    AIL_ID SysId,                        //in
    AIL_INT64 ProtocolType,              //in
    AIL_CONST_TEXT_PTR DeviceDescription, //in
    AIL_INT64 InitFlag,                  //in
    AIL_INT64 InitValue,                 //in
    AIL_ID *ComIdPtr                     //out
)
```

Description

This function allocates an Industrial Communication context on the specified system. An Industrial Communication context allows you to communicate with a robot controller, with a generic controller that uses the specified industrial protocol (PROFINET, Modbus, EtherNet/IP, or CC-Link IE Field Basic), or with a server Modbus automation device.

For a context allocated to communicate using the PROFINET, Modbus, EtherNet/IP, or CC-Link IE Field Basic protocol, a connection with the controller (or device) is established upon allocation of the context. As such, the protocol service must have been enabled and configured in Aurora Imaging Configurator prior to allocating the context.

For a context allocated to communicate with a robot controller, you can choose to set the InitFlag parameter to M_COM_NO_CONNECT so that you do not connect automatically upon allocation. If this option is chosen, you must use McomControl() with M_COM_ROBOT_CONNECT to manually establish the connection with the robot controller.

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

After allocating the Industrial Communication context, you should check if the operation was successful, using MappGetError() or by verifying that the Industrial Communication context identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the Industrial Communication context is no longer required, release it using McomFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the Industrial Communication context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the Industrial Communication context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.						a	h	i							
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().						a	h	i							

ProtocolType INQ

Specifies the type of protocol to use.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of protocol																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_PROTOCOL_ABB	Specifies an ABB robot controller protocol.						a	h	i							
M_COM_PROTOCOL_CCLINK	Specifies a CC-Link IE Field Basic protocol.						a	h	i							
M_COM_PROTOCOL_DENSO	Specifies a DENSO robot controller protocol.						a	h	i							
M_COM_PROTOCOL_EPSON	Specifies an Epson robot controller protocol.						a	h	i							
M_COM_PROTOCOL_ETHERNETIP	Specifies an Ethernet/IP protocol.						a	h	i							
M_COM_PROTOCOL_FANUC	Specifies a Fanuc robot controller protocol.						a	h	i							
M_COM_PROTOCOL_KUKA	Specifies a KUKA robot controller protocol.						a	h	i							
M_COM_PROTOCOL_MODBUS	Specifies a Modbus protocol.						a	h	i							
M_COM_PROTOCOL_PROFINET	Specifies a PROFINET protocol.						a	h	i							
	System specific															

		This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									
M_COM_PROTOCOL_STAUBLI		Specifies a Staubli robot controller protocol.						a	h	i							

DeviceDescription INQ

Specifies the description of the device with which to communicate. This parameter must be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For describing the device																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("M_DEFAULT") 1	Specifies to communicate with the default device described in Aurora Imaging Configurator that uses the specified protocol. If the protocol selected in the ProtocolType parameter is not enabled in the Aurora Imaging Configurator utility (for the non-robot controller types of Industrial Communication contexts), an error is returned.						a	h	i							
AIL_TEXT("*InstanceName*") 1	Specifies the instance name of a given Industrial Communication protocol. The name and other description/value pairs can be found on the **Communication** page of the Aurora Imaging Configurator utility on the runtime platform. Use the instance name when there is more than one instance of a given protocol listed in Aurora Imaging Configurator; otherwise, use "M_DEFAULT".						a	h	i							
AIL_TEXT("*IPOrName:Port*") 1	Specifies the IP address and port on the robot controller to use to communicate with it. The Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255. INQ						a	h	i							

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

InitFlag INQ

Specifies how to initialize the Industrial Communication context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of initialization																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default behavior. This is the only possible setting when allocating a context to communicate with a non-robot controller device.						a	h	i							
M_COM_NO_CONNECT	Specifies not to automatically connect to the robot controller.						a	h	i							

InitValue

Reserved for future expansion and must be set to **M_DEFAULT**.

ComIdPtr

Specifies the address of the variable in which to write the Industrial Communication context identifier or specifies the data type that the function should use to return the Industrial Communication context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Industrial Communication context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.						a	h	i							

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Industrial Communication context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_COM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Industrial Communication context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.						a	h	i							
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Industrial Communication context. If allocation fails, **M_NULL** is written as the identifier.						a	h	i							

Return value

The returned value is the Industrial Communication context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_COM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomControl

Synopsis

Control an Industrial Communication context setting.

Syntax

```
void McomControl(
    AIL_ID ComId,           //out
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function changes a setting of an Industrial Communication context.

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **McomControl()** internally calls the *AIL_DOUBLE* version of this function (McomControlDouble()). If you need to pass text values, call the *AIL_CONST_TEXT_PTR* version of this function (McomControlText()).

Parameters

ComId

Specifies the identifier of the Industrial Communication context.

ControlType

Specifies the Industrial Communication setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For any type of Industrial Communication context
- For an Industrial Communication context for use with a robot controller

The following ControlType and ControlValue parameter settings can be specified for any type of Industrial Communication context:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any type of Industrial Communication context																			
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_COM_TIMEOUT		Sets the maximum amount of time to wait for a receiving function (for example, McomRead() or McomWaitPositionRequest()) to finish its execution. INQ															
	Value	Specifies the maximum amount of time to wait, in msecs.						a	h	i							

The following ControlType and ControlValue parameter settings can be specified for an Industrial Communication context used to communicate with a robot controller:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an Industrial Communication context for use with a robot controller																			
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_COM_REMOTE_ADDRESS		Sets the IP address of a robot controller. If an IP address was specified upon context allocation (using McomAlloc() with "IPOrName:Port"), setting this control type will overwrite that address. Changing the value of this control type will have no effect until you call **McomControl**() with M_COM_ROBOT_CONNECT. If the Industrial Communication context was previously connected to a robot controller, you must first disconnect it from the robot controller using **McomControl**() with M_COM_ROBOT_DISCONNECT. INQ															
	AIL_TEXT("*n.n.n.n*") 1	Specifies the IP address of the robot controller. Express the address as a string in dotted decimal notation, where each dotted decimal value (*n*) is a number between 0 and 255. > Note, if using a C compiler, you must pass the value as M_PTR_TO_AIL_INT(AIL_TEXT("*n.n.n.n*")).						a	h	i							

M_COM_REMOTE_PORT	Sets the port on the robot controller to use to communicate with it. If a port number was specified upon context allocation (using McomAlloc() with "IPOrName:Port"), setting this control type will overwrite that port number. Changing the value of this control type will have no effect until you call **McomControl**() with M_COM_ROBOT_CONNECT. If the Industrial Communication context was previously connected to a robot controller, you must first disconnect it from the robot controller using **McomControl**() with M_COM_ROBOT_DISCONNECT. INQ						a	h	i						
Value > 0	Specifies the port on the robot controller.						a	h	i						
M_COM_ROBOT_CONNECT	Connects to the robot controller.						a	h	i						
M_DEFAULT	Specifies the default behavior.						a	h	i						
M_COM_ROBOT_CONNECT_RETRY	Sets the maximum number of times to attempt to reconnect to the robot controller. INQ						a	h	i						
Value	Specifies the maximum number of times to attempt to reconnect.						a	h	i						
M_COM_ROBOT_CONNECT_RETRY_WAIT	Sets the amount of time to wait between attempts to connect to the robot controller. INQ						a	h	i						
Value	Specifies the amount of time to wait, in msecs.						a	h	i						
M_COM_ROBOT_DISCONNECT	Disconnects from the robot controller.						a	h	i						
M_DEFAULT	Specifies the default behavior.						a	h	i						

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Type-specific versions of the function when using a C compiler under 64-bit

```
void McomControlText (AIL_ID ComId, AIL_INT64 ControlType, AIL_CONST_TEXT_PTR ControlValue)
```

Details

Parameters

ComId

See ComId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void McomControlDouble (AIL_ID ComId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ComId

See ComId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomFree

Synopsis

Free an Industrial Communication context.

Syntax

```
void McomFree(
    AIL_ID ComId //in
)
```

Description

This function frees an Industrial Communication context.

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

All Industrial Communication contexts allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ComId

Specifies the identifier of the Industrial Communication context.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomGetHookInfo

Synopsis

Get information about a communication hook event.

Syntax

```
AIL_INT McomGetHookInfo(
    AIL_ID EventId,     //in
    AIL_INT64 InfoType, //in
    void *UserVarPtr    //out
)
```

Description

This function retrieves information about the communication event that caused the hook-handler function to be called. **McomGetHookInfo**() should only be called within the scope of an Industrial Communication hook-handler function (McomHookFunction()).

Parameters

EventId

Specifies the identifier of the communication event received by the hook-handler function (see McomHookFunction()).

InfoType

Specifies the type of information about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- AIL_INT

Specifies the address of the variable in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **InfoType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring information

For inquiring information	
InfoType	Description
UserVarPtr - Possible values returned	

M_COM_ERROR_NUMBER		Retrieves the error code returned by the last Aurora Imaging Library communication function call.
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*
	M_ERROR_COM_NETWORK_ROBOT_CONNECT	Specifies a network error code. This error code is generated when Aurora Imaging Library cannot connect to the robot.
	Value	Specifies the Aurora Imaging Library communication error code.
M_COM_PROFINET_SLOT_CHANGED_COUNT		Retrieves the number of modules (slots) that were detected as changed.
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*
M_COM_PROFINET_SLOT_CHANGED_LIST		Retrieves a list containing the module numbers (slot numbers) of all the modules that changed.
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*

Return value

The returned value is **M_NULL** if successful. If the operation fails, a non-null (!**M_NULL**) value is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomHookFunction

Synopsis

Hooks a function to a communication event.

Syntax

```
void McomHookFunction(
    AIL_ID ComId,                           //in
    AIL_INT HookType,                       //in
    AIL_COM_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                       //in-out
)
```

Description

This function allows you to attach or detach a user-defined function to a specified communication event. Once a hook-handler function is defined and hooked to an event, it is automatically called when the event occurs. The events that can be hooked represent actions by the PLC or robot in the Aurora Imaging Library application.

You can hook more than one function to an event by making separate calls to **McomHookFunction**() for each function that you want to hook. Aurora Imaging Library automatically chains and keeps an internal list of all these hooked functions. When a function is hooked, this new function is added to the end of the list. When the event happens, all user-defined functions in the list will be executed in the same order that they were hooked to the event. You can also remove any function from the list; in this case, Aurora Imaging Library preserves the order of the remaining functions in the list.

You can obtain more information about the event from within the hook-handler function using McomGetHookInfo().

Parameters

ComId

Specifies the identifier of the industrial communication context. The context must have been allocated using McomAlloc() with M_COM_PROTOCOL_PROFINET.

HookType

Specifies the data change event to which to hook the function.

For specifying the data change event to hook	
Value	Description
M_COM_DATA_CHANGE +	Calls the hook-handler function when a change has been made to an output module. Note that this type of event is only supported when using the PROFINET protocol.
M_COM_ERROR_TRIGGER +	Calls the hook-handler function when an Aurora Imaging Library communication error occurs.

Combination value for the values listed in For specifying the data change event to hook.

You can add the following value to the above-mentioned values to specify that the function should be unhooked.

For specifying that the function should be hooked or unhooked	
Combination value	Description
M_UNHOOK	Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function.

HookHandlerPtr

Specifies the address of the function that should be called when the specified event occurs. The hook-handler function must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Specifies the type of communication event that generated the call.

EventId

Event identifier to pass to McomGetHookInfo() when inquiring about the hooked event.

UserDataPtr

Specifies the user data pointer passed to the UserDataPtr parameter of **McomHookFunction**().

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if not used.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomInquire

Synopsis

Inquire about an Industrial Communication context setting.

Syntax

```
AIL_INT McomInquire(
    AIL_ID ComId,          //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires about a setting of the Industrial Communication context.

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

Parameters

ComId

Specifies the identifier of the Industrial Communication context.

InquireType

Specifies the setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_BOOL
- AIL_ID
- AIL_INT
- AIL_INT64
- AIL_UINT64

Specifies the address of the variable in which to return the value of the inquired setting. Since the **McomInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about the system

- For any type of Industrial Communication context
- For an Industrial Communication context for use with a robot controller
- For an Industrial Communication context for use with a PROFINET device
- For an Industrial Communication context for use with a EtherNet/IP device
- For an Industrial Communication context for use with a CC-Link IE Field Basic device

The following InquireType parameter settings can be specified for any Industrial Communication context type, to inquire the system on which the context was allocated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the system																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_OWNER_SYSTEM		Inquires the identifier of the system on which the Industrial Communication context was allocated. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*						a	h	i							
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.						a	h	i							
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().						a	h	i							

The following InquireType parameter settings can be specified for any Industrial Communication context type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For any type of Industrial Communication context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_COM_DEVICE_DESCRIPTION +		Inquires the description of the device. SET															
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*						a	h	i							
	AIL_TEXT("*InstanceName*") 1	Specifies the instance name of a given Industrial Communication protocol. INFO						a	h	i							

	AIL_TEXT("*IPOrName:Port*") 1	Specifies the IP address and port on the robot controller to use to communicate with it. INFO						a	h	i						
	AIL_TEXT("M_DEFAULT") 1	Specifies to communicate with the default device described in Aurora Imaging Configurator that uses the specified protocol. INFO						a	h	i						
M_COM_GET_CONNECTION_STATE		Inquires whether the device is connected.						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*						a	h	i						
	M_FALSE	Specifies that the device is not connected.						a	h	i						
	M_TRUE	Specifies that the device is connected.						a	h	i						
M_COM_INIT_VALUE		Inquires how the Industrial Communication context was initialized. SET						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT64*						a	h	i						
	M_DEFAULT	Specifies the default behavior. INFO						a	h	i						
	M_COM_NO_CONNECT	Specifies not to automatically connect to the robot controller.						a	h	i						
M_COM_IS_HARDWARE		Inquires if the Aurora Imaging Library system is using a hardware-assisted acceleration device to communicate with the target device.						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*						a	h	i						
	M_FALSE	Specifies that the system is not using a hardware-assisted acceleration device.						a	h	i						
	M_TRUE	Specifies that the system is using a hardware-assisted acceleration device.						a	h	i						

M_COM_PROTOCOL_TYPE		Inquires the type of protocol for which the context was allocated. SET						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*						a	h	i						
	M_COM_PROTOCOL_ABB	Specifies an ABB robot controller protocol.						a	h	i						
	M_COM_PROTOCOL_CCLINK	Specifies a CC-Link IE Field Basic protocol.						a	h	i						
	M_COM_PROTOCOL_DENSO	Specifies a DENSO robot controller protocol.						a	h	i						
	M_COM_PROTOCOL_EPSON	Specifies an Epson robot controller protocol.						a	h	i						
	M_COM_PROTOCOL_ETHERNETIP	Specifies an Ethernet/IP protocol.						a	h	i						
	M_COM_PROTOCOL_FANUC	Specifies a Fanuc robot controller protocol.						a	h	i						
	M_COM_PROTOCOL_KUKA	Specifies a KUKA robot controller protocol.						a	h	i						
	M_COM_PROTOCOL_MODBUS	Specifies a Modbus protocol.						a	h	i						
	M_COM_PROTOCOL_PROFINET	Specifies a PROFINET protocol. INFO						a	h	i						
	M_COM_PROTOCOL_STAUBLI	Specifies a Staubli robot controller protocol.						a	h	i						
M_COM_TIMEOUT		Inquires the maximum amount of time to wait for a receiving function (for example, McomRead() or McomWaitPositionRequest()) to finish its execution. SET						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i						
	Value	Specifies the maximum amount of time to wait, in msecs.						a	h	i						

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

The following InquireType parameter settings can be specified for an Industrial Communication context allocated to communicate with a robot controller.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an Industrial Communication context for use with a robot controller																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_COM_REMOTE_ADDRESS +	Inquires the IP address of the robot controller. SET SET															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*						a	h	i							
AIL_TEXT("*n.n.n.n*") 1	Specifies the IP address of the robot controller.						a	h	i							
M_COM_REMOTE_PORT	Inquires the port on the robot controller to use to communicate with it. SET SET						a	h	i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							
Value > 0	Specifies the port on the robot controller.						a	h	i							
M_COM_ROBOT_CONNECT_RETRY	Inquires the maximum number of times to attempt to reconnect to the robot controller. SET						a	h	i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							
Value	Specifies the maximum number of times to attempt to reconnect.						a	h	i							
M_COM_ROBOT_CONNECT_RETRY_WAIT	Inquires the amount of time to wait between attempts to connect to the robot-controller. SET						a	h	i							
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							

	Value	Specifies the amount of time to wait, in msecs.						a	h	i							
M_COM_ROBOT_ISCONNECTED		Inquires whether the robot controller is connected.						a	h	i							
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*						a	h	i							
	M_FALSE	Specifies the robot controller is not connected.						a	h	i							
	M_TRUE	Specifies the robot controller is connected.						a	h	i							

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for M_COM_DEVICE_DESCRIPTION; M_COM_REMOTE_ADDRESS.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0").															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							

The following InquireType parameter settings can be specified for an Industrial Communication context allocated to communicate using the PROFINET protocol (McomAlloc() with M_COM_PROTOCOL_PROFINET).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an Industrial Communication context for use with a PROFINET device																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_COM_PROFINET_GET_MODULE_INSIZE +	Inquires the amount of memory that the PROFINET controller expects to be allocated to the module in local memory, which can be used to send input data to the controller (for example, a PLC). The expected module's size can be modified using the Siemens Totally Integrated Automation (TIA) portal of the controller. Note that if the PROFINET controller is not currently connected, the module's last known size is returned. You must specify a combination value from the following table: • To specify the module\'s index in the local computer															
	System specific															
	This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							
Value	Specifies the expected input module's size, in bytes.						a	h	i							

M_COM_PROFINET_GET_MODULE_OUTSIZE +	Inquires the amount of memory that the PROFINET controller expects to be allocated to the module in local memory, to which the controller can write. The expected module's size can be modified using the Siemens Totally Integrated Automation (TIA) portal of the controller. Note that if the PROFINET controller is not currently connected, the module's last known size is returned. You must specify a combination value from the following table: • To specify the module\'s index in the local computer						a	h	i							
	System specific															
	This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							
Value	Specifies the expected output module's size, in bytes.						a	h	i							
M_COM_PROFINET_GET_PLC_STATE	Inquires the state of the PROFINET controller (PLC) connected to the local computer. The inquired controller state is the state as of the last alarm transfer cycle.						a	h	i							
	System specific															

		This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							
	M_COM_PROFINET_STATUS_RUN +	Specifies that the PROFINET PLC is running.						a	h	i							
	M_COM_PROFINET_STATUS_STOP +	Specifies that the PROFINET PLC has stopped.						a	h	i							

Combination values for [when InquireType=M_COM_PROFINET_GET_PLC_STATE: M_COM_PROFINET_STATUS_RUN; M_COM_PROFINET_STATUS_STOP].

One of the following values might be combined with the above-mentioned values to specify in which redundant mode the PLC is running.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a PROFINET PLC in a redundant mode																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_PROFINET_STATUS_BACKUP	Specifies that the PROFINET PLC, running in a redundant mode, is using the backup network channel.						a	h	i							
	System specific															
	This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									
M_COM_PROFINET_STATUS_PRIMARY	Specifies that the PROFINET PLC, running in a redundant mode, is using the primary network channel.						a	h	i							
	System specific															

		This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									

Combination value for M_COM_PROFINET_GET_MODULE_INSIZE; M_COM_PROFINET_GET_MODULE_OUTSIZE.

You must add the following value to the above-mentioned values to specify the module's index in the local computer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To specify the module's index in a PROFINET automation device																	
Combination value	Description		(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
0 <= *Index* <= 15	Specifies the module's index in the local computer.							a	h	i							
		System specific															
		This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									

The following InquireType parameter settings can be specified for an Industrial Communication context allocated to communicate using the EtherNet/IP protocol (McomAlloc() with M_COM_PROTOCOL_ETHERNETIP).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an Industrial Communication context for use with a EtherNet/IP device																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_COM_ETHERNETIP_CONFIG_SIZE		Inquires the amount of memory that is allocated to the configuration assembly in local memory, from which the controller can read EtherNet/IP configuration data that you have written about the local computer. Note that the size of this assembly can be changed using the Aurora Imaging Configurator utility. The controller must be configured to recognize this size.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i						
	Value	Specifies the size of the configuration assembly, in bytes.						a	h	i						
M_COM_ETHERNETIP_CONSUMER_SIZE		Inquires the amount of memory that is allocated to the consumer assembly in local memory, to which the controller can write. Note that the size of this assembly can be changed using the Aurora Imaging Configurator utility. The controller must be configured to recognize this size.						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i						
	Value	Specifies the size of the consumer assembly, in bytes.						a	h	i						
M_COM_ETHERNETIP_PRODUCER_SIZE		Inquires the amount of memory that is allocated to the producer assembly in local memory, from which the controller can read. Note that the size of this assembly can be changed using the Aurora Imaging Configurator utility. The controller must be configured to recognize this size.						a	h	i						
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i						

	Value	Specifies the size of the producer assembly, in bytes.						a	h	i							

The following InquireType parameter settings can be specified for an Industrial Communication context allocated to communicate using the CC-Link IE Field Basic protocol (McomAlloc() with M_COM_PROTOCOL_CCLINK).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an Industrial Communication context for use with a CC-Link IE Field Basic device																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_COM_CCLINK_TOTAL_OCCUPIED_STATIONS		Inquires the number of stations occupied by the CC-Link IE Field Basic protocol instance. This determines the size of the register and flag data tables available for the controller to read from and write to. For each station, there are 32 input registers, 32 output registers, 64 input flags, and 64 output flags. Note that the number of stations occupied by the protocol instance can be changed using the Aurora Imaging Configurator utility. The controller must be configured to make this number of stations available.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*						a	h	i							
	Value	Specifies the number of stations.						a	h	i							

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomRead

Synopsis

Read data from a device that uses the specified communication protocol.

Syntax

```
void McomRead(
    AIL_ID ComId,                          //in
    AIL_CONST_TEXT_PTR DataObjectEntryName, //in
    AIL_INT Offset,                        //in
    AIL_INT Size,                          //in
    void *UserArrayPtr                     //out
)
```

Description

This function reads data from a device that uses the specified communication protocol. In most cases, this function actually reads the local computer's memory for data received from a controller (e.g. a PLC) during the controller's last write cycle. However, when communicating with the device using the Modbus protocol in client configuration, or reading configuration data from an EtherNet/IP device explicitly, this function can read the data directly from the device's memory. This function can also read the data last written to local memory using McomWrite().

A controller that uses the PROFINET protocol writes data to the local computer in modules in local memory. As such, you must specify the module's index from which to read the received data.

A client that uses the Modbus protocol writes data to the local computer in data tables in local memory, when the local computer is in a server configuration. To read from a Modbus device (either in a server or client configuration), you must specify the data table from which to read.

A client that uses the EtherNet/IP protocol writes data to the local computer, typically, in the consumer assembly in local memory. As such, to read data received from an EtherNet/IP client, specify to read from the consumer assembly in local memory. You can also read from the producer or configuration assembly in local memory. When reading from a producer assembly, you are reading your own published data in local memory. When reading from a configuration assembly, you are reading EtherNet/IP configuration data in local memory about the local computer. Note that you can also specify another device on the network from which to read data explicitly, although this should only be used to read configuration data.

A controller that uses the CC-Link IE Field Basic protocol writes data to the local computer, in register and flag data tables in local memory. As such, to read data received from a CC-Link controller, you must specify whether to read the data in the flags or registers data table. The local computer can only be configured as a worker device.

For PROFINET and EtherNet/IP, use McomInquire() with M_COM_PROFINET_GET_MODULE_OUTSIZE or M_COM_ETHERNETIP_CONSUMER_SIZE to establish the size of the output module, or assembly, respectively, to which the controller has written. You must know how the module/assembly is divided into fields.

For CC-Link IE Field Basic, use McomInquire() with M_COM_CCLINK_TOTAL_OCCUPIED_STATIONS to learn the number of stations used by the industrial protocol instance. To establish the number of registers (RWw) or flags (RY) to which the controller can write, multiply the number of stations by 32 or 64

respectively.

If the time to execute this function is greater than the value set using McomControl() with M_COM_TIMEOUT, an error occurs.

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

Parameters

ComId

Specifies the identifier of the Industrial Communication context that specifies the protocol of the device with which to communicate.

DataObjectEntryName

Specifies the location from which to read the data. This location is protocol specific. This parameter must be set to one of the following.

When reading from a device (controller) that uses the PROFINET protocol (McomAlloc() with M_COM_PROTOCOL_PROFINET), use the following format to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to read in PROFINET																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*mcom://ModuleIndex*") [1]	Specifies the location in local memory from which to read the data received from a controller. Replace *ModuleIndex* with the index of the module in local memory from which to read the data. *ModuleIndex* can be from 0 to 255.						a	h	i							
	System specific															
	This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When reading from a device (client or server automation device) that uses the Modbus protocol (McomAlloc() with M_COM_PROTOCOL_MODBUS), use the following format to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to read in Modbus

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COILS	Specifies to read the coil data table in local memory for data received from a Modbus client, or written by the last call to McomWrite() with M_COILS.						a	h	i							
M_DISCRETE_INPUT	Specifies to read the discrete input table in local memory for data written by the last call to McomWrite() with M_DISCRETE_INPUT.						a	h	i							
M_HOLDING_REGISTER	Specifies to read the holding register table in local memory for data received from a Modbus client, or written by the last call to McomWrite() with M_HOLDING_REGISTER.						a	h	i							
M_INPUT_REGISTER	Specifies to read the input register table in local memory for data written by the last call to McomWrite() with M_INPUT_REGISTER.						a	h	i							

AIL_TEXT("*mcom://ServerId[@ServerIP]/M_COILS*") 1	Specifies to read the coil data table of a server, when the local computer is communicating using the Modbus protocol in client configuration. Replace *ServerId* with the server's identifier number. If connecting to a server over the Modbus TCP protocol, also enter @ and replace *[ServerIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255.						a	h	i						
AIL_TEXT("*mcom://ServerId[@ServerIP]/M_DISCRETE_INPUT*") 1	Specifies to read the discrete input data table of a server, when the local computer is communicating using the Modbus protocol in client configuration. Replace *ServerId* with the server's identifier number. If connecting to a server over the Modbus TCP protocol, also enter @ and replace *[ServerIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255.						a	h	i						

AIL_TEXT("*mcom://ServerId[@ServerIP]/M_HOLDING_REGISTER*") [1]	Specifies to read the holding register data table of a server, when the local computer is communicating using the Modbus protocol in client configuration. Replace *ServerId* with the server's identifier number. If connecting to a server over the Modbus TCP protocol, also enter @ and replace *[ServerIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255.						a	h	i						
AIL_TEXT("*mcom://ServerId[@ServerIP]/M_INPUT_REGISTER*") [1]	Specifies to read the input register data table of a server, when the local computer is communicating using the Modbus protocol in client configuration. Replace *ServerId* with the server's identifier number. If connecting to a server over the Modbus TCP protocol, also enter @ and replace *[ServerIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255.						a	h	i						

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When reading from a device (typically a client) that uses the EtherNet/IP protocol (McomAlloc() with M_COM_PROTOCOL_ETHERNETIP), or when reading local memory for data written by the last call to

McomWrite(), use one of the following to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to read in EthernetIP																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_ETHERNETIP_CONFIG	Specifies to read the configuration assembly in local memory for EtherNet/IP configuration data typically written by the last call to McomWrite().						a	h	i							
M_COM_ETHERNETIP_CONSUMER	Specifies to read the consumer assembly in local memory for data received from an EtherNet/IP client.						a	h	i							
M_COM_ETHERNETIP_PRODUCER	Specifies to read the producer assembly in local memory for data written by the last call to McomWrite().						a	h	i							
AIL_TEXT("*mcom://[RemoteIP/]AssemblyID*") [1]	Specifies another server on the network and the assembly from which to read the data when communicating using the Ethernet/IP protocol. Replace *[RemoteIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255. This method uses explicit messaging to read from a particular device's specified assembly. While this is supported in Aurora Imaging Library, this method will be slower than using implicit messaging. This should, in theory, only be used to read a configuration setting on the specified device.						a	h	i							

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When reading from a device (controller) that uses the CC-Link IE Field Basic protocol (McomAlloc() with M_COM_PROTOCOL_CCLINK), or when reading local memory for data written by the last call to McomWrite(), use one of the following to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to read in CC-Link IE Field Basic																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_CCLINK_INPUT_FLAG	Specifies to read the input flag (RX) data table in local memory for data written by the last call to McomWrite().						a	h	i							
M_COM_CCLINK_INPUT_REGISTER	Specifies to read the input flag (RWr) data table in local memory for data written by the last call to McomWrite().						a	h	i							
M_COM_CCLINK_OUTPUT_FLAG	Specifies to read the output flag (RY) data table in local memory for data received from a CC-Link IE Field Basic controller.						a	h	i							
M_COM_CCLINK_OUTPUT_REGISTER	Specifies to read the output register (RWw) data table in local memory for data received from a CC-Link IE Field Basic controller.						a	h	i							

Offset

Specifies the offset from which to start reading the data in the specified module/data table/assembly. Typically, the offset is specified in bytes. The following exceptions apply:

- For Modbus register-type data tables (M_..._REGISTER) and CC-Link register-type data tables (M_COM_CCLINK_..._REGISTER), specify the offset in 16-bit registers.
- For Modbus bit-type data tables (M_COILS and M_DISCRETE_INPUT) and CC-Link flag-type data tables (M_COM_CCLINK_..._FLAG), specify the offset in bits.

Size

Specifies the amount of the data to read. Typically, the size is specified in bytes. The following exceptions apply:

- For Modbus register-type data tables (M_..._REGISTER) and CC-Link register-type data tables (M_COM_CCLINK_..._REGISTER), specify the size in 16-bit registers.
- For Modbus bit-type data tables (M_COILS and M_DISCRETE_INPUT) and CC-Link flag-type data tables (M_COM_CCLINK_..._FLAG), specify the size in bits.

UserArrayPtr
Accepts the following address:

- Data type: address of an AIL_UINT8
  Required array size:(Size)

Specifies the address of the array in which to write the data.

When reading 1-bit data, the array contains one byte for each bit that is read. The read values are stored in the least-significant bit of each byte.

When reading 16-bit data, the array contains two bytes for each 16-bit word that is read. The read values are stored as byte pairs. The first byte is the most-significant byte of the word, the second byte is the least-significant byte of the word.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomSendPosition

Synopsis

Sends a position to the robot. This does not move the robot, but simply sends a found position to the robot controller.

Syntax

```
void McomSendPosition(
    AIL_ID ComId,              //in
    AIL_INT64 OperationCode,   //in
    AIL_INT64 Status,          //in
    AIL_INT64 ObjectSpecifier, //in
    AIL_DOUBLE PositionX,      //in
    AIL_DOUBLE PositionY,      //in
    AIL_DOUBLE PositionZ,      //in
    AIL_DOUBLE RotationX,      //in
    AIL_DOUBLE RotationY,      //in
    AIL_DOUBLE RotationZ,      //in
    AIL_INT64 ControlFlag,     //in
    AIL_DOUBLE ControlValue    //in
)
```

Description

This function sends a position to the robot controller. It is typically used to change the robot's position to the specified coordinates.

You can only use this function when you specify an Industrial Communication context that has been allocated to communicate with a robot controller.

Use RotationX, RotationY, RotationZ to specify rotations about the different positional axes. The table below indicates how the robot controllers internally represent this data.

Robot type	Robot controller representation		
	RotationX	RotationY	RotationZ
ABB	The orientation is converted by the robot controller to a quaternion.		
DENSO	Roll	Pitch	Yaw
Epson	W	V	U

Fanuc	W	P	R
KUKA	C	B	A
Staubli	Rx	Ry	Rz

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

Parameters

ComId

Specifies the identifier of the Industrial Communication context. The Industrial Communication context must have been allocated to communicate with a robot controller.

OperationCode

Specifies the operation code (opcode) of the command. This parameter must be set one of to the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation code																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_ROBOT_FIND_POSITION_RESULT	Specifies that the Aurora Imaging Library application is sending a new position to the robot controller.						a	h	i							
1024 <= *Value* <= 2047	Specifies the value of the user-defined operation code. Note that if you want to start the user operation code offset at 0 instead of 1024, you can use **M_COM_USER_OPCODE** + opcode value.						a	h	i							

Status

Specifies the status code to send to the robot controller.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the status code sent to the robot controller																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COM_ERROR	Specifies that the Aurora Imaging Library application was not successful in finding a new position or that the robot should stop.						a	h	i								
M_COM_SUCCESS	Specifies that the Aurora Imaging Library application was successful in finding a new position.						a	h	i								

ObjectSpecifier

Specifies the user identifier associated with the object at the specified position. This should ideally correspond to the value received from ObjectSpecifierPtr during the previous call of the McomWaitPositionRequest() function.

PositionX

Specifies the X-position to the robot controller.

PositionY

Specifies the Y-position to the robot controller.

PositionZ

Specifies the Z-position to the robot controller.

RotationX

Specifies the rotation around the positional X-axis to the robot controller.

RotationY

Specifies the rotation around the positional Y-axis to the robot controller.

RotationZ

Specifies the rotation around the positional Z-axis to the robot controller.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ControlValue

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.

McomWaitPositionRequest

Synopsis

Waits to receive a robot's request for a new position.

Syntax

```
void McomWaitPositionRequest(
    AIL_ID ComId,                  //in
    AIL_INT64 *OperationCodePtr,   //out
    AIL_INT64 *StatusPtr,          //out
    AIL_INT64 *ObjectSpecifierPtr, //out
    AIL_DOUBLE *PositionXPtr,      //out
    AIL_DOUBLE *PositionYPtr,      //out
    AIL_DOUBLE *PositionZPtr,      //out
    AIL_DOUBLE *RotationXPtr,      //out
    AIL_DOUBLE *RotationYPtr,      //out
    AIL_DOUBLE *RotationZPtr,      //out
    AIL_INT64 ControlFlag,         //in
    AIL_DOUBLE ControlValue        //in
)
```

Description

This function waits to receive a robot's request for a new position. When a robot controller sends a request for a new position, it also sends it's current position and other relevant information.

You can only use this function when you specify an Industrial Communication context that has been allocated to communicate with a robot controller.

If the time to execute this function is greater than the value set using McomControl() with M_COM_TIMEOUT, an error occurs.

The robot's current rotation about the position axes is received in the RotationXPtr, RotationYPtr, and RotationZPtr parameters. The table below indicates how the robot controller internally represents this data.

Robot Type	Robot controller representation		
	RotationXPtr	RotationYPtr	RotationZPtr
ABB	The orientation is converted by the robot controller to a quaternion.		
DENSO	Roll	Pitch	Yaw
Epson	W	V	U

Fanuc	W	P	R
KUKA	C	B	A
Staubli	Rx	Ry	Rz

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

Parameters

ComId

Specifies the identifier of the Industrial Communication context. The Industrial Communication context must have been allocated to communicate with a robot controller.

OperationCodePtr

Specifies the address of the variable in which to write the operation code (opcode) received from the robot controller. This parameter will return the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation code																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_FIND_POSITION_REQUEST	Specifies that the robot controller requests a new position.						a	h	i							
1024 <= *Value* <= 2047	Specifies the robot controller requests to perform a user-defined operation, identified by the operation code. Note that you can compare the value returned to the specified address with a 0-based opcode using **M_COM_USER_OPCODE** + 0-based opcode. For example, an opcode of 1026 is equivalent to **M_COM_USER_OPCODE** + 2.						a	h	i							

StatusPtr

Specifies the address of the variable in which to write the status code received from the robot controller. This status code indicates whether the robot was successful in moving/using the last specified position.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the status code returned by the robot controller

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COM_ERROR	Specifies that the robot controller returns a failure status.						a	h	i							
M_COM_SUCCESS	Specifies that the robot controller returns a success status.						a	h	i							

ObjectSpecifierPtr

Specifies the address of the variable in which to write the user identifier associated with the object on which to perform the specified operation; the identifier is established and received from the robot-side software. This is typically used for applications with multiple object types or multiple occurrences of objects.

PositionXPtr

Specifies the address of the variable in which to write the current X-position of the robot.

PositionYPtr

Specifies the address of the variable in which to write the current Y-position of the robot.

PositionZPtr

Specifies the address of the variable in which to write the current Z-position of the robot.

RotationXPtr

Specifies the address of the variable in which to write the rotation around the positional X-axis of the robot.

RotationYPtr

Specifies the address of the variable in which to write the rotation around the positional Y-axis of the robot.

RotationZPtr

Specifies the address of the variable in which to write the rotation around the positional Z-axis of the robot.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ControlValue

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.

DLL	Requires ail11.dll; ailcom11.dll.

McomWrite

Synopsis

Write data to a device that uses the specified communication protocol.

Syntax

```
void McomWrite(
    AIL_ID ComId,                          //in
    AIL_CONST_TEXT_PTR DataObjectEntryName, //in
    AIL_INT Offset,                        //in
    AIL_INT Size,                          //in
    const void *UserArrayPtr               //in
)
```

Description

This function writes data to a device that uses the specified communication protocol. In most cases, this function actually writes to the local computer's memory, the data to send to a controller (e.g. a PLC) during the controller's next read cycle. However, when communicating with the device using the Modbus protocol in client configuration, or writing configuration data to an EtherNet/IP device explicitly, this function can write the data directly to the device's memory.

A controller that uses the PROFINET protocol reads data from the local computer, from modules in local memory. As such, you must specify the module's index in which to write the data to send to the controller.

A client that uses the Modbus protocol reads data from the local computer, from data tables in local memory, when the local computer is in a server configuration. To write to a Modbus device (either in a server or client configuration), you must specify the data table in which to write. Note that when the local computer is configured as a client, you can only write to the coil or holding register data tables of a server.

A client that uses the EtherNet/IP protocol reads data from the local computer, from a producer assembly in local memory. As such, to send data to an EtherNet/IP client, specify to write to the producer assembly in local memory. You can also write to a configuration assembly. When writing to a configuration assembly, you are writing EtherNet/IP configuration data in local memory about the local computer.

> For EtherNet/IP, you can also specify to write explicitly to another device on the network. However, this method is slower than using implicit messaging. You should only use explicit messaging for data that is not time critical; typically, this means only writing data to the configuration assemblies of other devices.

A controller that uses the CC-Link IE Field Basic protocol reads data from the local computer, from register and flag data tables in local memory. As such, to send data to a CC-Link controller, you must specify whether to write the data in the flags or registers data table. The local computer can only be configured as a worker device.

For PROFINET and EtherNet/IP, use McomInquire() with M_COM_PROFINET_GET_MODULE_INSIZE, M_COM_ETHERNETIP_PRODUCER_SIZE to establish the size of the input module, or assembly, respectively, from which the controller can read. You must know how the module/assembly is divided into fields.

For CC-Link IE Field Basic, use McomInquire() with M_COM_CCLINK_TOTAL_OCCUPIED_STATIONS to

learn the number of stations used by the industrial protocol instance. To establish the number of registers (RWr) or flags (RX) from which the controller can read, multiply the number of stations by 32 or 64 respectively.

> This function is only available if you have the Aurora Imaging Library Industrial & Robot Communications package, or another relevant update installed.

Parameters

ComId

Specifies the identifier of the Industrial Communication context that specifies the protocol of the device with which to communicate.

DataObjectEntryName

Specifies the location in which to write the data. This location is protocol specific. This parameter must be set to one of the following.

When writing to a device (controller) that uses the PROFINET protocol (McomAlloc() with M_COM_PROTOCOL_PROFINET), use the following format to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to write in PROFINET																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*mcom://ModuleIndex*") [1]	Specifies the location in local memory in which to write the data to send to a controller. Replace *ModuleIndex* with the index of the input module in local memory in which to write the data. *ModuleIndex* can be from 0 to 255.						a	h	i							
	System specific															
	This constant is only available on a Host system if the Host system was previously allocated on a Zebra 4Sight EV6/EV7. It is not available on Zebra 4Sight XV6/XV7.						a									

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When writing to a device (client) that uses the Modbus protocol (McomAlloc() with M_COM_PROTOCOL_MODBUS), use the following format to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to write in Modbus																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COILS	Specifies to write to the coils data table in local memory the data to send to the Modbus client.						a	h	i						
M_DISCRETE_INPUT	Specifies to write to the discrete input data table in local memory the data to send to the Modbus client.						a	h	i						
M_HOLDING_REGISTER	Specifies to write to the holding register data table in local memory the data to send to the Modbus client.						a	h	i						
M_INPUT_REGISTER	Specifies to write to the input register data table in local memory the data to send to the Modbus client.						a	h	i						

AIL_TEXT("*[mcom://ServerId[@ServerIP]/]M_COILS*") 1	Specifies to write to the coils data table of a server, when the local computer is communicating using the Modbus protocol in client configuration. Replace *ServerId* with the server's identifier number. If connecting to a server over the Modbus TCP protocol, also enter @ and replace *[ServerIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255.						a	h	i						

AIL_TEXT("*[mcom://ServerId[@ServerIP]/]M_HOLDING_REGISTER*") [1]	Specifies to write to the holding register data table of a server, when the local computer is communicating using the Modbus protocol in client configuration. Replace *ServerId* with the server's identifier number. If connecting to a server over the Modbus TCP protocol, also enter @ and replace *[ServerIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255.						a	h	i							

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When writing to a device (typically a client) that uses the EtherNet/IP protocol (McomAlloc() with M_COM_PROTOCOL_ETHERNETIP), use one of the following to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to write in EtherNet/IP																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COM_ETHERNETIP_CONFIG	Specifies to write EtherNet/IP configuration data about the local computer to the configuration assembly in local memory.						a	h	i							
M_COM_ETHERNETIP_PRODUCER	Specifies to write data to the producer assembly in local memory to send the data to the Ethernet/IP client.						a	h	i							
AIL_TEXT("*mcom://[RemoteIP]/AssemblyID*") [1]	Specifies the server on the network and the assembly in which to write the data when communicating using the Ethernet/IP protocol. Replace *[RemoteIP]* with the server's IP address; the Aurora Imaging Library Industrial Communication module supports IPv4 addresses. A typical IPv4 string has the format n.n.n.n, where n is a number between 0 and 255. This method uses explicit messaging to write to a particular device's specified assembly. While this is supported in Aurora Imaging Library, this method will be slower than using implicit messaging. Therefore, in theory, this method should only be used to write configuration data to another server on the network.						a	h	i							

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

When writing to a device (controller) that uses the CC-Link IE Field Basic protocol (McomAlloc() with M_COM_PROTOCOL_CCLINK), use one of the following to specify the location.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the path of the data to write in CC-Link IE Field Basic																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COM_CCLINK_INPUT_FLAG	Specifies to write the data to the input flag (RX) data table in local memory to send the data to the CC-Link IE field basic controller.						a	h	i							
M_COM_CCLINK_INPUT_REGISTER	Specifies to write the data to the input register (RWr) data table in local memory to send the data to the CC-Link IE field basic controller.						a	h	i							

Offset

Specifies the offset from which to start writing the data in the specified module/data table/assembly. Typically, the offset is specified in bytes. The following exceptions apply:

- For Modbus register-type data tables (M_..._REGISTER) and CC-Link register-type data tables (M_COM_CCLINK_INPUT_REGISTER), specify the offset in 16-bit registers.
- For Modbus bit-type data tables (M_COILS and M_DISCRETE_INPUT) and CC-Link flag-type data tables (M_COM_CCLINK_INPUT_FLAG), specify the offset in bits.

Size

Specifies the size of the data to be written. Typically, the size is specified in bytes. The following exceptions apply:

- For Modbus register-type data tables (M_..._REGISTER) and CC-Link register-type data tables (M_COM_CCLINK_INPUT_REGISTER), specify the size in 16-bit registers.
- For Modbus bit-type data tables (M_COILS and M_DISCRETE_INPUT) and CC-Link flag-type data tables (M_COM_CCLINK_INPUT_FLAG), specify the size in bits.

UserArrayPtr
Accepts the following address:

- Data type: address of an AIL_UINT8
  Required array size:(Size)

Specifies the address of the array containing the data.

When writing 1-bit data, the array must contain one byte for each bit that is read. The values to write must be stored in the least-significant bit of each byte.

When writing 16-bit data, the array must contain two bytes for each 16-bit word that is written. The values to write must be stored as byte pairs. The first byte is the most-significant byte of the word, the second byte is the least-significant byte of the word.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcom11.lib.
DLL	Requires ail11.dll; ailcom11.dll.